LensesTemplate_v3

October 11, 2026

1 Lenses Analysis File

[18]:
```
#Import pandas, numpy, and matplotlib

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

1.0.1 *Before you begin your data analysis remember that you need to upload your data file to the local directory in Jupyter.*

1.1 Lens Thickness

Import the appropriate sheet from your excel file to work with the lens thickness data.

Syntax: *datafile = pandas.read_excel('FileName.xlsx', sheet_name = #)*

[11]:
```
import pandas as pd

#Import data
datafile = pd.read_excel('Optics_All_Raw_Measurements.xlsx',
                         sheet_name = 'Lens thickness readings')

#Display data
display(datafile)
```

```
                                                Trial  d (cm)  d (cm)  \
0                                                   1    75.6   65.35
1                                                   2    75.6   65.48
2                                                   3    75.6   65.40
3                                                   4    75.6   65.45
4                                                   5    75.6   65.35
5                                                 NaN     NaN     NaN
6        Source: IMG_1284(1).jpeg; IMG_1282(1).jpeg     NaN     NaN
7  Raw transcription only. No calculated quantities.     NaN     NaN

   d (cm)                                Transcription note
0   66.05                          Verify overwritten d .
1   66.15              Verify d ; d  continued by arrow.
```

```
2    66.15  Parenthetical d  used; verify correction. d  c…
3    66.25                              d  continued by arrow.
4    66.15                              d  continued by arrow.
5      NaN                                                 NaN
6      NaN                                                 NaN
7      NaN                                                 NaN
```

Convert the columns of an excel table into vectors by calling the column header of a given column.

Syntax: *xdata = datafile['ColumnHeader'].to_numpy()*

Once the data is converted, determine the lens thickness by taking the difference between the two measurements.

[72]:
```
#Convert data from list to array
#xdata = datafile['d  (cm)'].to_numpy()
#ydata = datafile['d  (cm)'].to_numpy()

xdata = datafile['d  (cm)'].iloc[:5].to_numpy()
ydata = datafile['d  (cm)'].iloc[:5].to_numpy()

n = len(xdata)
#print(n)
def findT(n, xdata, ydata):

    t = np.zeros(n)
    total = 0
    for i in range(n):
        t[i] = ydata[i] - xdata[i]
    for i in range(n):
        total += t[i]
    return total/n, t

#Determine thickness of the lens (calculating the average)
LensThickness = findT(n, xdata, ydata)
print("The average lens thicknes is: ", LensThickness)
```

```
The average lens thicknes is:  (np.float64(0.7440000000000027), array([0.7 ,
0.67, 0.75, 0.8 , 0.8 ]))
```

Calculate the average thickness of the lens by averaging the entries in the LensThickness array. You can do this by using built in functions that are available in the Numpy library. Note that the sample standard deviation is with the argument ddof=1. Ref: https://numpy.org/doc/stable/reference/generated/numpy.std.html

Syntax: *Average = numpy.mean(Array)*

Syntax: *StandardDeviation = numpy.std(Array, ddof=1)*

You will have to calculate the standard deviation of the mean by writing out the equation of the standard deviation of the mean. There is no built in function for it.

$\sigma_m = \frac{\sigma_{std}}{\sqrt{N}}$ You can determine N (number of trials) by taking the length of the array you are taking the average of.

Syntax: *LengthofArray = len(Array)*

[40]:
```
#Calc average thickness
AverageLensThickness = np.mean(t)
LensThicknessStd = np.std(t, ddof=1)
AverageLensThicknessErr = LensThicknessStd/n**0.5

#Print values
print(AverageLensThickness)
print(LensThicknessStd)
print(AverageLensThicknessErr)
```

```
0.5840000000000003
0.33019691094860343
0.14766854776830435
```

1.2 Object at Infinity

Using the data for object at infinity and the tips above for determining the average and its uncertainty, calculate the focal length of the lens.

[73]:
```
#Import data
datafile2 = pd.read_excel('Optics_All_Raw_Measurements.xlsx',
                          sheet_name = 'Object at infinity')

#Display data
display(datafile2)
```

	Trial	d (mm), ±0.5 mm	\
0	1	62.0	
1	2	569.0	
2	3	405.0	
3	4	237.0	
4	5	604.0	
5	NaN	NaN	
6	Source: IMG_1292(1).jpeg; IMG_1285(1).jpeg	NaN	
7	Raw transcription only. No calculated quantities.	NaN	

	d (mm), ±0.5 mm	Transcription note
0	272.0	NaN
1	775.0	Verify d ; crossed-out 61 under d excluded.
2	272.0	NaN
3	508.0	NaN
4	815.0	Verify d .
5	NaN	NaN
6	NaN	NaN
7	NaN	NaN

[74]:
```
#Convert data from list to array
print(datafile.columns.tolist())

xdata2 = datafile2['d  (mm), ±0.5 mm'].iloc[:5].to_numpy()
ydata2 = datafile2['d  (mm), ±0.5 mm'].iloc[:5].to_numpy()
n2 = len(xdata2)
#Calc average focal length

AverageFocalLength, t2 = findT(n, xdata2, ydata2)
FocalLengthStd = np.std(t2, ddof=1)
AverageFocalLengthErr = AverageFocalLength/n**0.5

#Print values
print(AverageFocalLength)
print(FocalLengthStd)
print(AverageFocalLengthErr)
```

```
['Trial', 'd  (cm)', 'd  (cm)', 'd  (cm)', 'Transcription note']
153.0
162.12803582354286
68.42368011149357
```

1.3 Thin Lens Equation

[75]:
```
#Import data
datafile3 = pd.read_excel('Optics_All_Raw_Measurements.xlsx',
                          sheet_name = 'Thin lens')

#Display data
display(datafile3)
```

```
                                                Entry  d  (cm)  d  (cm)  \
0                                                   1      9.5    48.50
1                                                   2      9.5    55.85
2                                                   3      9.5    58.10
3                                                   4      9.5    60.90
4                                                 NaN      NaN      NaN
5                           Source: IMG_1286(1).jpeg      NaN      NaN
6  Raw transcription only. No calculated quantities.      NaN      NaN

   d  (cm)                                Transcription note
0     50.3                  Verify d  and d  handwriting.
1     51.5                         Verify d  handwriting.
2     52.2                                            NaN
3     52.5  Bottom table transcribed; repeated working abo…
4      NaN                                            NaN
5      NaN                                            NaN
6      NaN                                            NaN
```

[76]:
```
#Convert data from list to array
zdata3 = datafile3['d  (cm)'].iloc[:4].to_numpy()
xdata3 = datafile3['d  (cm)'].iloc[:4].to_numpy()
ydata3 = datafile3['d  (cm)'].iloc[:4].to_numpy()
```

Using the Thin Lens equation (Eq 12), plot a graph of inverse object distance vs inverse image distance. Using linearization, determine the focal length of the lens.

Reminder: Use notes from the OpAmp excerise to help with plotting.

To determine the uncertainties on the x-variable and y-variables, you will need to use propagation of uncertainty.

[81]:
```
#Define a figure so that you can plot your line of best fit over your data␣
 ↪later in the file

n3 = len(xdata3)

imageAverage, image_distances = findT(n3, xdata3, zdata3)
objectAverage, object_distances = findT(n3, ydata3, xdata3)

#print("Image distances:", image_distances)
#print("Object distances:", object_distances)
#print("n3:", n3)


x = 1 / image_distances
y = 1 / object_distances
#Plot a graph with both x and y error bars present (they may be two small to␣
 ↪see). Provide a label for the the legend.


m, b = np.polyfit(x, y, 1)

# Points for drawing the fitted line
xline = np.linspace(x.min(), x.max(), 100)

plt.scatter(x, y, label="Measurements")
plt.plot(xline, m*xline + b, label="Linear fit")



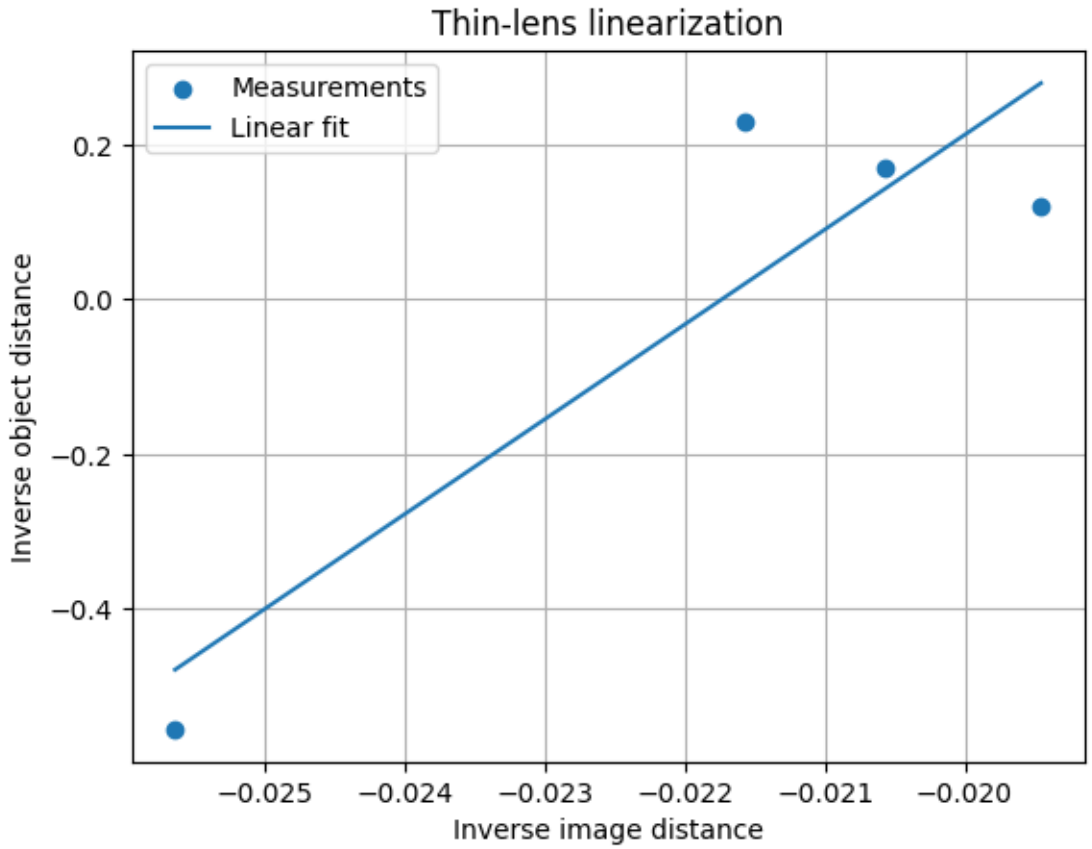
#x-axis title
#y-axis title
#Figure title


plt.xlabel("Inverse image distance")
```

```
plt.ylabel("Inverse object distance")
plt.title("Thin-lens linearization")
plt.legend()
plt.grid(True)
plt.show()

# Determine focal length from the intercept
f = 1 / b
print("Focal length:", f)

print("Slope:", m, "Intercept:", b)
print("Object distances:", object_distances)
print("Image distances:", image_distances)
```




```
Focal length: 0.3753715896486225
Slope: 122.56228692017356 Intercept: 2.66402686718001
Object distances: [-1.8   4.35  5.9   8.4 ]
Image distances: [-39.   -46.35 -48.6  -51.4 ]
```

Instead of doing the linear fit by hand using the Least Squares Method, we can use a built in fit function through Numpy called polyfit.

Syntax: *FitParameters, CovMatrix = numpy.polyfit(xVar, yVar, PolynomialDegree, cov = True)*

The polyfit function will fit polynomials of nth degrees. There are four input parameters that we need to worry about for this course:

1. The x-variable array (*xVar*)
2. The y-varibale array (*yVar*)
3. The degree of polynomial we want to fit (*PolynomialDegree*). For a linear fit this would be a first degree polynomial, this would be indicated with a 1.
4. Lastly we want to be able to determine the uncertainies on our fit parameters, so we need to set the output for the covarience matrix to be True (*cov = True*).

There are two outputs from the fit that we want to assign to arrays:

1. The fit parameters (*FitParameters*). This will store the slope and the y-intercept from the fit into an array.
2. The covarience Matrix (*CovMatrix*). This will store values of covarience in an array. From this, we can determine the uncertainies on our fit parameters by square rooting the variance terms of the slope and y-intercept (the diagonal elements).

To create an array of the the diagonal elements of the covarience matrix for calculating the uncertainty of the slope and y-intercept we can use the Numpy function (diag). We can the take the square root of this and the result will be the the uncertainties.

Syntax: *FitParametersErr = numpy.sqrt(numpy.diag(CovMatrix))*

The last thing we need to do is assign the parameters from the fit array (and uncertainty array) to individual varaibles. If we assign variables seperated by commas, python with iteratively assign these varaibles through an array. This allows us to assign the slope and y-intercept at the same time.

Syntax: *slope, yInt = FitParameters*

For addition information about the Polyfit function, please see the documentation: https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html

[85]:
```
#Use polyfit to complete the fit
FitParameters, CovMatrix = np.polyfit(x, y, 1, cov=True)
FitParametersUncertainty = np.sqrt(np.diag(CovMatrix))

#Assign parameters from fit
m, b = FitParameters
m_err, b_err = FitParametersUncertainty

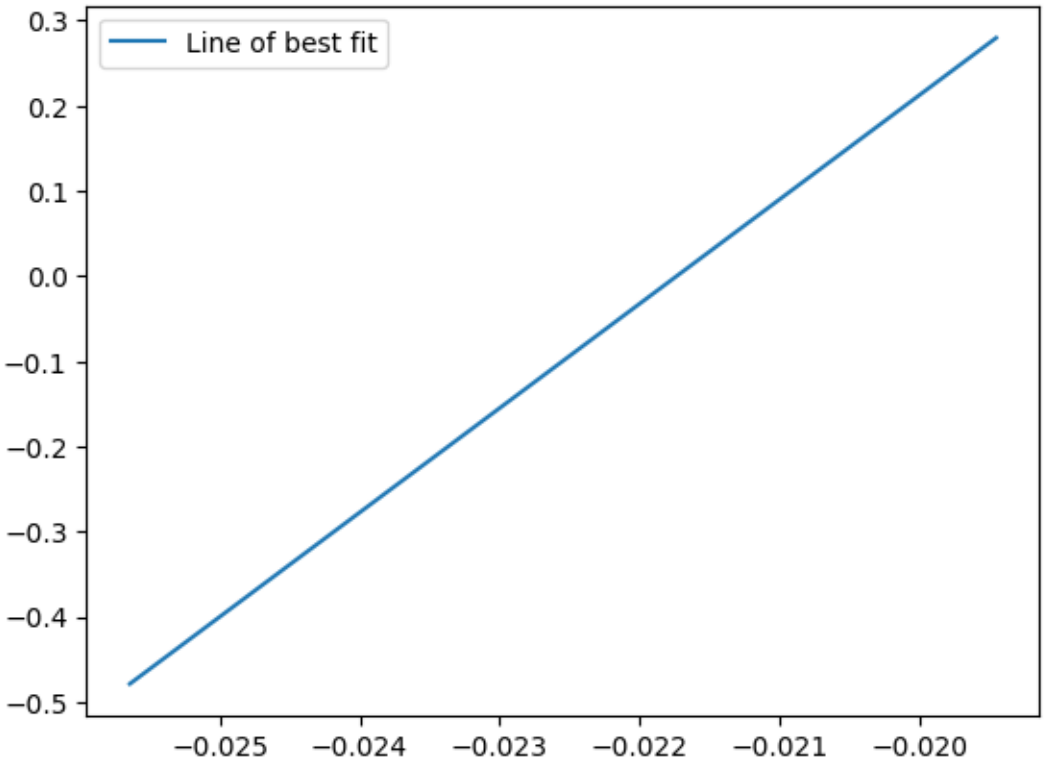
#Print values
print("Here is m:", m)
print("Here is b:", b)
print("Here is error on m:", m_err)
print("Here is error on b:", b_err)
```

```
Here is m: 122.56228692017356
Here is b: 2.66402686718001
Here is error on m: 41.91217971709768
Here is error on b: 0.919403088757004
```

[87]:
```
#Determine y-values for the lines of best fit using fit parameters
y_fit = m * x + b

#plot line of best fit on the same figure as your data. Again, provide a label␣
 ↪for the legend.
plt.plot(x, y_fit, label="Line of best fit")
# Add a legend
plt.legend()

# Display the figure
plt.show()
```



[88]:
```
#Save a copy of the graph.
plt.savefig("thin_lens_graph.png", dpi=300, bbox_inches="tight")
```

```
<Figure size 640x480 with 0 Axes>
```

[90]:
```
#Calculating error in y...
ruler_err = 0.05  # cm

#Combining independent errors in quadrature...
object_distance_err = np.sqrt(ruler_err**2 + ruler_err**2)

#Getting y_err through error propagation...
y_err = object_distance_err / object_distances**2

#Calculate chi squared
chi_squared = np.sum(((y - y_fit) / y_err)**2)


#Print chi squared
print("Chi squared:", chi_squared)
```

```
Chi squared: 29005.795653579138
```

Relating your linear fit to the Thin Lens Equation (Eq 12), determine the focal length of your lens. Use propogation of error to determine the uncertainty.

[91]:
```
#Determine the focal lenth of the lens using the fit parameters
ThinLensFocal =  1 / b

#Propogation of error
# First-order propagation of uncertainty
ThinLensFocalErr = b_err / b**2

#Print focal length
print("Focal length:", ThinLensFocal, "+/-", ThinLensFocalErr)
```

```
Focal length: 0.3753715896486225 +/- 0.1295474168096107
```

1.3.1 For the remainder of the file follow the prompts to complete the analysis. Please refer to explainations above for help with averaging and graphing.

1.4 Mirror Technique

[92]:
```
#Import data
datafile4 = pd.read_excel('Optics_All_Raw_Measurements.xlsx',
                          sheet_name = 'Mirror')
#Display data
display(datafile4)
```

```
                                          Entry  d (cm)  d (cm)  \
0                                             1     9.4    25.7
1                                           NaN     NaN     NaN
2                          Source: IMG_1287(1).jpeg     NaN     NaN
3  Raw transcription only. No calculated quantities.     NaN     NaN
```

```
   d  (unit not shown)                                    Transcription note
0                50.65  Verify d  and d ; d  is marked "don't need" in…
1                  NaN                                                NaN
2                  NaN                                                NaN
3                  NaN                                                NaN
```

[93]:
```
#Convert data from list to array
xdata4 = datafile4['d  (cm)'].iloc[:1].to_numpy()
ydata4 = datafile4['d  (cm)'].iloc[:1].to_numpy()

#Calc average focal length
avgFocalLength = 25.7 - 9.4

#Print values
print(avgFocalLength)
```

```
16.299999999999997
```

1.5 Lens Makers Equation

[95]:
```
#Import data
datafile5 = pd.read_excel('Optics_All_Raw_Measurements.xlsx',
                          sheet_name = 'Spherometer')

#Display data
display(datafile5)
```

```
                                                Entry  r  (mm)  r  (mm)  \
0                                                   1     1.16     1.15
1                                                   2     1.16     1.16
2                                                   3     1.16     1.16
3                                                   4     1.16     1.16
4                                                   5     1.16     1.16
5                                                 NaN      NaN      NaN
6                           Source: IMG_1283(1).jpeg      NaN      NaN
7  Raw transcription only. No calculated quantities.      NaN      NaN
8  Material label appears as "BK2 (glass)"; verif…      NaN      NaN

   l  (mm)  Calibration (mm)  Recorded l  uncertainty (mm)  \
0    22.58             -0.13                         NaN
1    21.57             -0.13                         NaN
2    22.07             -0.13                        0.02
3      NaN             -0.13                         NaN
4      NaN             -0.13                         NaN
5      NaN               NaN                         NaN
6      NaN               NaN                         NaN
7      NaN               NaN                         NaN
8      NaN               NaN                         NaN
```

```
                                Transcription note
0                              Verify l  first row.
1                                                NaN
2  Uncertainty explicitly written beside this ent…
3                               l  blank in source.
4                               l  blank in source.
5                                                NaN
6                                                NaN
7                                                NaN
8                                                NaN
```

[105]:
```
#Convert data from list to array
rs1 = datafile5['r  (mm)'].iloc[:5].to_numpy()
rs2 = datafile5['r  (mm)'].iloc[:5].to_numpy()
tot1 = 0
tot2 = 0
tot3 = 0

#Determine the average measured distance values from the spherometer on side 1
for i in range(5):
    tot1 += rs1[i] - 0.13
thick1 = tot1/5
print("The average of side 1 is: ", thick1)

#Determine the average measured distance values from the spherometer on side 2
for i in range(5):
    tot2 += rs2[i] - 0.13
thick2 = tot2/5
print("The average of side 2 is: ", thick2)

#Determine the average measured distance between the feet of the spherometer
ls = datafile5['l  (mm)'].iloc[:3].to_numpy()
for i in range(3):
    tot3 += ls[i]
feets = tot3/3
print("The average of feets distances is: ", feets)
```

```
The average of side 1 is:  1.0299999999999998
The average of side 2 is:  1.0279999999999998
The average of feets distances is:  22.073333333333334
```

[115]:
```
#Calculate the radius of curveture for side 1 of the lens (use propogation of␣
 ↪error to determine the uncertainty)
sigma_ds = 0.02 # mm feets
sigma_rs = 0.005 # mm thick

Radius1 = (feets**2)/2*thick1 + 0.5*thick1
```

```
Radius1Uncertainty = np.sqrt(
    (feets / thick1)**2 * sigma_ds**2
    + (-0.5 * feets**2 / thick1**2 + 0.5)**2 * sigma_rs**2
)
print("The radius 1 is: ", Radius1)
print("The uncertainty on the radius 1 is: ", Radius1Uncertainty)


#Calculate the radius of curveture for side 2 of the lens (use propogation of␣
 ↪error to determine the uncertainty)
Radius2 = (feets**2)/2*thick2 + 0.5*thick2
Radius2Uncertainty = np.sqrt(
    (feets / thick2)**2 * sigma_ds**2
    + (-0.5 * feets**2 / thick2**2 + 0.5)**2 * sigma_rs**2
)

print("The radius 2 is: ", Radius2)
print("The uncertainty on the radius 2 is: ", Radius2Uncertainty)


#Index of refraction for BK7 glass
n = 1.5168

#Calculate focal length and uncertainty using the Lens Makers Equation (Eq 14)
#Since both lenses are convex and light is passing from one side, one will␣
 ↪appear concave. Don't forget the negative sign.
LenMakerFocalLength = ((n-1)*(1/Radius1 - 1/Radius2))**(-1)
LenMakerFocalLengthUncertainty =  1 / ((n - 1) * Radius1**2 * (1/Radius1 - 1/
 ↪Radius2)**2)

#Print focal length
print("The focal length is : ", LenMakerFocalLength)
print("The focal uncertainty is : ", LenMakerFocalLengthUncertainty)
```

```
The radius 1 is:  251.43950288888882
The uncertainty on the radius 1 is:  1.2232072146297028
The radius 2 is:  250.95127084444442
The uncertainty on the radius 2 is:  1.2276880233695255
The focal length is :  -250077.21456056635
The focal uncertainty is :  511215.1702787314
```

1.6 Magnification

[116]:
```
#Import data
datafile6 = pd.read_excel('Optics_All_Raw_Measurements.xlsx',
                          sheet_name = 'Magnification')
```

```
#Display data
display(datafile6)
```

```
                                                Entry   s  (mm)   s  (mm)   \
0                                                   1    16.21     8.75
1                                                   2    17.11    32.31
2                                                   3    19.11    16.71
3                                                   4    15.63    17.76
4                                                   5    17.10    10.62
5                                                 NaN      NaN      NaN
6                             Source: IMG_1288(1).jpeg      NaN      NaN
7  Raw transcription only. No calculated quantities.       NaN      NaN

                                  Transcription note
0                           Verify s  first digit.
1                                              NaN
2  Verify s  replacement below crossed-out value.
3                                              NaN
4                                              NaN
5                                              NaN
6                                              NaN
7                                              NaN
```

[131]:
```
#Convert data from list to array
objSizes = datafile6['s  (mm)'].iloc[:5].to_numpy()
imgSizes = datafile6['s  (mm)'].iloc[:5].to_numpy()
objmean = np.mean(objSizes)
imgmean = np.mean(imgSizes)
readingerror = 0.02 # mm
#Determine the inverse of the magnification (y-varibale) and uncertainty using␣
 ↪propogation of error. The magnification can be determined using Eq 16.
#InverseMagnification = (-imgmean/objmean)**(-1)
InverseMagnification = (-objmean/imgmean)
InverseMagnificationUncertainty = np.sqrt((-1/imgmean)**2*readingerror**2
                                          +(objmean/imgmean**2*readingerror**2))

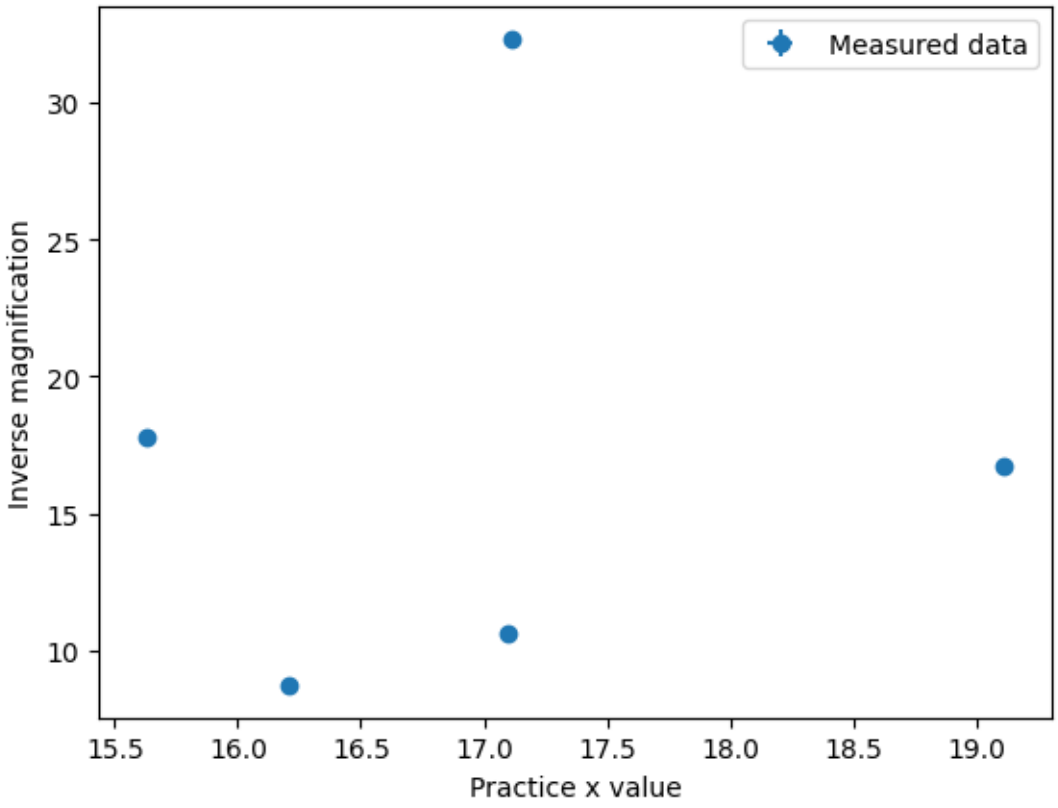
#Define a figure so that you can plot your line of best fit over your data␣
 ↪later in the file
fig, ax = plt.subplots()
#Plot a graph with both x and y error bars present (they may be two small to␣
 ↪see). Provide a label for the the legend.
ax.errorbar(
    objSizes, imgSizes,
    xerr=readingerror,
    yerr=readingerror,
    fmt="o",
    label="Measured data"
```

```
)

ax.legend()


#x-axis title
#y-axis title
#Figure title

ax.set_xlabel("Practice x value")
ax.set_ylabel("Inverse magnification")
ax.legend()
plt.show()
```



[135]:
```
#Use polyfit to complete the fit
np.polyfit(x, y, 1)

#Assign parameters from fit
a, b = np.polyfit(x, y, 1)
#Print values
print("Slope a =", a)
```

```
print("Intercept b =", b)
```

```
Slope a = 122.56228692017356
Intercept b = 2.66402686718001
```

[137]:
```
#Determine y-values for the lines of best fit using fit parameters
y_fit = a * x + b

#plot line of best fit on the same figure as your data. Again, provide a label␣
 ↪for the legend.
ax.plot(x, y_fit, label="Line of best fit")
ax.legend()
#add a legend
#print figure

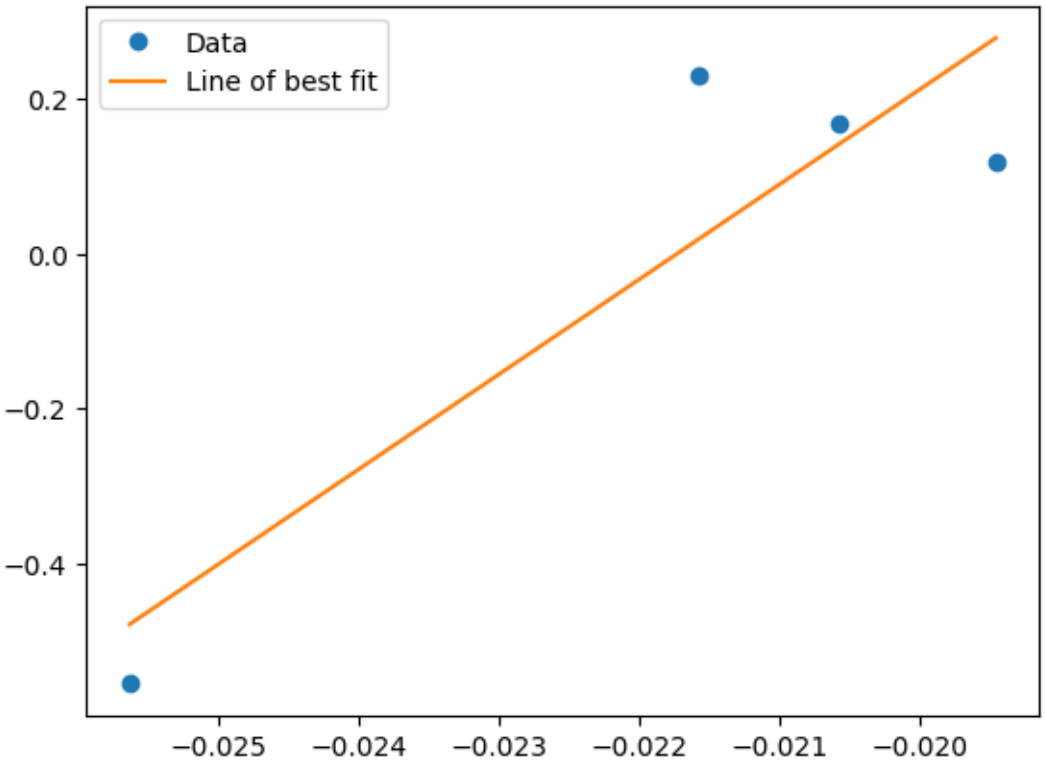
# Add a legend
ax.legend()

# Display figure
plt.show()
```

[138]:
```
# Create figure and plot your data
fig, ax = plt.subplots()
ax.plot(x, y, "o", label="Data")

# Calculate and plot the best-fit line
a, b = np.polyfit(x, y, 1)
y_fit = a * x + b
ax.plot(x, y_fit, label="Line of best fit")

# Display the legend and graph
ax.legend()
plt.show()
```



[139]:
```
#Save a copy of the graph.
fig.savefig("best_fit_graph.png", dpi=300, bbox_inches="tight")
```

[140]:
```
#Calculate chi squared
chi_squared = np.sum(((y - y_fit) / y_err)**2)


#Print value
print("Chi squared:", chi_squared)
```

```
Chi squared: 29005.795653579138
```

Relating your linear fit to Equation 18 in the lab manual, determine the focal length of your lens. Use propogation of error to determine the uncertainty.

[177]:
```
#Calculate focal length using fit parameters
MagnificationFocalLength =
MagnificationFocalLengthUncertainty =

#Print focal length
#We did this part wrong, we did not measure S. Hence could not analyze this␣
 ↪part.
```